CS247 Assignment 3 Report

1. Overview

High-level data flow:

```
loadData()
   ├─ precompute gradient_field (central differences, once per dataset)
   ├─ marchingSquares()  → 2D contour VBO
   └─ marchingCubes()    → 3D surface VBO (positions + normals)

key (I/O/K/L)
   ├─ current_iso_value mutates
   ├─ marchingSquares() (re-extracts 2D contour)
   └─ marchingCubes()   (re-extracts 3D surface, only here, never per frame)

render3D() (every frame)
   ├─ unlit wireframe bounding box
   └─ drawSurface(): Gouraud-lit iso-surface triangles
```

The render loop never re-extracts geometry. It only re-issues a draw call against the buffer that was filled last time the iso value changed.

2. Requirements coverage

#	Requirement (from `README.md`)	Points	Where & how
1	Draw the iso surface for the selected iso value	25	`marchingCubes()` walks every cell, uses the supplied `edgeTable3D` and `triangleTable` to emit triangles. `drawSurface()` issues a single `glDrawArrays(GL_TRIANGLES, …)`.
2	Don't re-extract on every frame, only on iso value change	10	`marchingCubes()` is called only from `loadData()` and the four iso-value key handlers (`I`, `O`, `K`, `L`). The per-frame `render3D()` never calls it.
3	Compute exact triangle vertex positions by interpolation	15	Each crossed edge is interpolated `t = (iso − vA)/(vB − vA)`, then `pos = (1-t)*posA + t*posB`.
[illegible]	Normals from c[illegible] interpolated along edges	[illegible]	`gradientField` [illegible]
[illegible]	Iso value [illegible]	[illegible]	[illegible] mutate `current_iso_value` and [illegible] both `marchingSquares()` and `marchingCubes()`. [illegible] voxel centre [illegible]
Total		**100**	

Bonuses (Phong shading in the fragment shader, user-controllable light) are not implemented.

3. Marching Cubes walk-through

3.1 Per-cell case index

The volume is traversed as a `(dimX−1) × (dimY−1) × (dimZ−1)` grid of cube cells. The eight corner offsets follow the standard Lorensen and Cline convention used by the supplied lookup tables:

```
static const int cornerOffset[8][3] = {
    {0,0,0},{1,0,0},{1,1,0},{0,1,0},   // v0..v3 (bottom face, z)
    {0,0,1},{1,0,1},{1,1,1},{0,1,1}    // v4..v7 (top face,   z+1)
};
```

Each corner's intensity is compared to the iso value with an epsilon margin (degeneracy guard), and an 8-bit case index is assembled:

```
int cubeIdx = 0;
for (int c = 0; c < 8; ++c)
    if (vC[c] > iso + eps) cubeIdx |= (1 << c);
```

`cubeIdx ∈ [0, 255]`. `edgeTable3D[cubeIdx]` is a 12-bit mask of which of the cube's 12 edges the iso surface crosses. If it is zero the cell is skipped.

3.2 Exact triangle vertices (15 pts) and normals (15 pts)

For every crossed edge, the implementation runs the same interpolation on two quantities at once: the spatial position and the gradient that becomes the surface normal:

```
const float t = (iso − vA) / (vB − vA);    // (denom guarded with eps)

// position: voxel centre NDC of the two corners, lerped by t
pos.x = pA.x + t * (pB.x − pA.x);
pos.y = pA.y + t * (pB.y − pA.y);
pos.z = pA.z + t * (pB.z − pA.z);

// normal: central-difference gradient at each corner, lerped by t,
// then normalised, then flipped so it points outward (gradient points
// toward higher-density tissue; the surface normal should point away).
nx = gA.x + t * (gB.x − gA.x);
ny = gA.y + t * (gB.y − gA.y);
nz = gA.z + t * (gB.z − gA.z);
len = sqrt(nx² + ny² + nz²);
n   = -vec3(nx, ny, nz) / len;
```

The 12 resulting `(pos, normal)` pairs are written into per-cell scratch arrays (`ePos[12]`, `eNrm[12]`), then `triangleTable[cubeIdx]` is read to emit up to 5 triangles, each referencing three of those edges.

3.3 Gradient field, precomputed once (central differences, 15 pts)

The gradient is independent of iso value, so the field is computed once per dataset load in `loadData()`:

```
gx = 0.5 * ( V(x+1, y, z) - V(x-1, y, z) );    // interior
gy = 0.5 * ( V(x, y+1, z) - V(x, y-1, z) );
gz = 0.5 * ( V(x, y, z+1) - V(x, y, z-1) );
```

Boundary voxels use forward or backward differences instead of central, so the lookup `gradAt(x, y, z)` never reads out of bounds. On the head dataset this is a 184 × 256 × 170 × 3 = ~24 M floats (94 MB) one-time allocation, which then makes every later iso value change pay only for the marching cubes traversal.

3.4 Per-frame avoidance (15 pts)

`marchingCubes()` is called from three places only:

Trigger	When
`loadData()`	When the user picks a new dataset (`1`, `2`).
Key `I` / `O`	When `current_iso_value` is increased by 0.01/0.1.
Key `K` / `L`	When `current_iso_value` is decreased by 0.01/0.1.

The per-frame `render3D()` path is just:

```
glBindVertexArray(surfaceVAO);
glDrawArrays(GL_TRIANGLES, 0, surface_vertex_count);
```

No extraction work happens there. Slice and axis changes (`W` / `S` / `A`) deliberately do not call `marchingCubes()`. They only update the 2D contour.

3.5 Gouraud shading (15 pts)

`shaders/volume3D.vs` computes lighting per vertex, after applying the model transform:

```
mat3 normalMat = transpose(inverse(mat3(model)));
vec3 N         = normalize(normalMat * vertexNorm);
vec3 L         = normalize(lightPos - worldPos.xyz);

float ambient  = 0.18;
float diffuse  = max(dot(N, L), 0.0);
fragColor      = vertexColor * lightColor * (ambient + diffuse);
```

The fragment shader is intentionally trivial. It just outputs the interpolated `fragColor`, so the shading you see across each triangle is the rasteriser's linear interpolation of three vertex colours, i.e. the textbook definition of Gouraud shading.

A `useLighting` uniform toggles this off so the same shader can also draw the unlit wireframe bounding box (rendered as `vertexColor` without the lighting term).

4. Marching Squares (Assignment 2 reuse)

Per the user's instruction, the 2D contour code is identical to the Assignment 2 implementation. Same `edgeTable2D[16]`, same voxel centre NDC mapping, same epsilon aware classification, same saddle resolution that wraps each above corner with its own arc. See the Assignment 2 report (sections 3.1 to 3.5) for the full description.

The only structural change is that the iso value `I` / `O` / `K` / `L` keys now drive both `marchingSquares()` (cheap, ~10 ms) and `marchingCubes()` (expensive, hundreds of ms on the head dataset), which is why the program prints `marching cubes: N triangles` after every iso change.

5. Build & run

```
cd Assignment_3
cmake -S . -B build
cmake --build build -j
./build/assignment
```

Two windows open. At startup both are blank. Press **1** for `lobster.dat` (fast, ~87k triangles) or **2** for `skewed_head.dat` (~590k triangles, takes 1 to 3 s to extract).

6. Controls

Key	Action
`1`	Load `data/lobster.dat`
`2`	Load `data/skewed_head.dat`
`A`	Cycle viewing axis (only affects the 2D window)
`W` / `S`	Next / previous slice on the current axis
`I` / `O`	Iso value `+0.01` / `+0.1`. Re-runs marching cubes
`K` / `L`	Iso value `−0.01` / `−0.1`. Re-runs marching cubes
`C`	Toggle 2D contour overlay
`B`	Cycle background colour
`Esc`	Quit

3D window mouse interaction:

Mouse	Action
Left-drag	Rotate the volume
`Ctrl` + Left-drag (X)	Zoom out / in
`Home`	Reset to default view

The terminal prints `marching cubes: N triangles` every time the surface is rebuilt, which is convenient for telling the algorithm is doing something before the GPU shows the result.

7. Files of interest

File	Role
`src/CS247_prog.cpp`, `marchingCubes()` and `drawSurface()`	The 3D iso surface extractor and its renderer.
`src/CS247_prog.cpp`, `loadData()` gradient block	Central difference gradient precomputation.
`src/CS247_prog.h`	Global state, plus the supplied `edgeTable3D[256]` and `triangleTable[256][16]` lookups.
`shaders/volume3D.vs`	Gouraud shading and `useLighting` toggle.
`shaders/volume3D.fs`	Pass through (interpolation happens in the rasteriser).